Clinic Phone Agent — 小诊所 AI 电话前台（骨架）

Clinic Phone Agent — an AI phone receptionist for small clinics (skeleton)

接电话 → 回答诊所常见问题 → 预约 / 改约 / 取消 → 处理不了的留言或转真人 → 通话结束后写摘要给前台。

Answer the call → answer common clinic questions → book / reschedule / cancel appointments → take a message or transfer to a human when it can't handle something → write a post-call summary for the front desk.

> 这是一个**能跑、带测试的骨架**，没做优化。先读第 1–3 节理解"这事行业里怎么做"，再看第 4 节怎么跑。
>
> This is a **working skeleton with tests**, not optimized yet. Read sections 1–3 first to see how the industry approaches this, then section 4 for how to run it.

---

1. 行业里一般怎么做（三条路）

How the industry does it (three routes)

这个问题定义得很清楚，行业里基本就三种做法：

The problem is well defined, and the industry basically has three ways to solve it:

路线 Route	代表 Examples	你要做的事 What you build	适合 Best for
A. 直接买成品 **A. Buy an off-the-shelf product**	各种 "AI receptionist for dental/medical" 的垂直 SaaS Vertical SaaS products, "AI receptionist for dental/medical"	配置 + 对接 Configuration + integration	诊所自己用，不想要技术团队 A clinic using it directly, with no tech team
B. 托管语音 Agent 平台 **B. Hosted voice-agent platform**	Vapi、Retell AI、Bland、Synthflow Vapi, Retell AI, Bland, Synthflow	写 prompt、把"查空档 / 预约"做成 webhook 给平台调用 Write the prompt; expose "check availability / book" as webhooks the platform calls	代理公司 / 外包最常见，上线最快 Most common for agencies / contractors; fastest to launch

路线 Route	代表 Examples	你要做的事 What you build	适合 Best for
C. 自己搭 **C. Build it yourself**	Twilio（电话）+ OpenAI Realtime（语音模型），或者 Pipecat / LiveKit Agents 框架 Twilio (telephony) + OpenAI Realtime (voice model), or the Pipecat / LiveKit Agents frameworks	自己负责音频桥接、工具调用、日志 You own audio bridging, tool calling and logging	想掌控数据、成本、合规，或者要做成产品 When you want control over data, cost and compliance, or are building a product

不管选哪条，**架构其实都一样**，只是哪层是你写、哪层是买：

Whichever route you pick, **the architecture is the same**; the only difference is which layers you build and which you buy:

```
电话网络（PSTN）
Phone network (PSTN)
   │
电话运营商 / 号码（Twilio）          ← B 路线里平台帮你包了
Carrier / phone number (Twilio)    ← handled by the platform in route B
   │  音频流 / audio stream
语音模型（speech-to-speech：OpenAI Realtime；
         或者三段式：STT → LLM → TTS）        ← B 路线里平台帮你包了
Voice model (speech-to-speech: OpenAI Realtime;
         or a three-stage pipeline: STT → LLM → TTS)   ← handled by the platform in route B
   │  工具调用 / tool calls (function calling)
你的业务后端：排班规则、患者、预约、留言    ← 不管哪条路线，这层都得你写
Your business backend: scheduling rules, patients,
appointments, messages             ← you build this layer on every route
   │
诊所现有系统（EHR / 预约软件）或者我们自己的 DB
The clinic's existing system (EHR / booking software) or our own DB
```

这个仓库走的是 C 路线，理由如下：

This repo takes route C, for these reasons:

- 你已经用过 OpenAI Realtime，最熟。
  You've already used OpenAI Realtime, so it's what you know best.
- 面试要的是"能不能做"，自己搭能证明你理解每一层。之后真要快速交付，换成 B（Vapi/Retell）也只是把"语音那层"换掉，`app/scheduling.py` + `app/tools.py` **这层原封不动**——它们本来就是 webhook 能直接调的普通函数。
  The interview is about "can you build it", and building it yourself proves you understand every layer. If you later need to ship fast, switching to B (Vapi/Retell) only replaces the voice layer —

`app/scheduling.py` + `app/tools.py` **stay exactly as they are**, because they're already plain functions a webhook can call.

- 体量小：一两百个病人，一天大概十几到几十通电话。**一台小服务器 + SQLite/Postgres 就够了**，不需要队列、K8s、微服务。
  The scale is small: one or two hundred patients, roughly a dozen to a few dozen calls a day. **One small server + SQLite/Postgres is enough** — no queues, no K8s, no microservices.

关于"一两百个客户"——面试时一定要问清楚

About "one or two hundred clients" — be sure to clarify this in the interview

- 如果是**一家诊所有一两百个病人** → 就是这个仓库现在的单租户形态。
  If it means **one clinic with one or two hundred patients** → that's the single-tenant setup this repo has now.
- 如果是**这家公司服务一两百家诊所** → 要做多租户：`clinic.json` 改成数据库里的 `clinics` 表，按被叫号码（Twilio 的 `To` 参数）找到对应的诊所配置/prompt/数据。代码改动不大，但架构讨论的重点就不一样了。
  If it means **the company serves one or two hundred clinics** → it needs to be multi-tenant: turn `clinic.json` into a `clinics` table in the database, and look up each clinic's config / prompt / data by the number that was called (Twilio's `To` parameter). The code change is small, but the architecture discussion shifts focus.

---

2. 关于 RAG：这个体量不需要

About RAG: **not needed** at this scale

小诊所的"知识"就是地址、停车、营业时间、取消政策、保险……几页纸，几千字。 **直接全塞进 system prompt** 比 RAG 更好：没有检索延迟（电话里每 300ms 都能听出来）、不会漏检、prompt cache 友好。

A small clinic's "knowledge" is its address, parking, opening hours, cancellation policy, insurance… a few pages, a few thousand words. **Putting all of it directly in the system prompt** beats RAG: no retrieval latency (on a phone call every 300 ms is audible), nothing gets missed in retrieval, and it's prompt-cache friendly.

`app/knowledge.py` 已经做了自动切换：知识库 < 12k 字符时内联进 prompt；超过以后自动给模型加一个 `search_clinic_info` 工具（用的是很小的 BM25，零依赖）。真到几百份文档再换 embedding + pgvector。

`app/knowledge.py` already switches automatically: when the knowledge base is under 12k characters it's inlined into the prompt; above that, the model automatically gets a `search_clinic_info` tool (a tiny BM25, zero dependencies). Only at hundreds of documents would we move to embeddings + pgvector.

真正需要"检索"的是结构化数据——空档、预约、患者——这些靠**工具调用查数据库**，不是 RAG。

What really needs "retrieval" is structured data — open slots, appointments, patients — and that's handled by **tool calls that query the database**, not RAG.

3. 架构和关键设计（面试时讲这些）

Architecture and key design decisions (talk about these in the interview)

```
来电 / Incoming call ──> Twilio 号码 / number ──POST /twilio/voice──> 返回 / returns TwiML
<Connect><Stream>
                │
                └── wss /twilio/media-stream  (G.711 μ-law 8kHz)
                          │   app/telephony/twilio_stream.py
                          │   (音频桥 + 打断处理 / audio bridge + barge-in handling)
                          │
                          └── wss OpenAI Realtime  (app/realtime_agent.py:
                                    │   会话配置、工具循环、转写 /
                                    │   session config, tool loop, transcription)
                                    │ function call
                                    v
                            app/tools.py  (工具定义 + 分发，身份绑定在通话上下文 /
                                           tool definitions + dispatch; identity bound to
the call context)
                                    │
                            app/scheduling.py  (纯业务规则，不碰 LLM / pure business rules,
no LLM)
                                    │
                            app/db.py  (SQLite → Postgres)
通话结束 / Call ends ──> app/call_lifecycle.py: 存转写 + 工具调用记录 + LLM 摘要 →
/admin/calls 给前台看
                         saves transcript + tool-call log + LLM summary → /admin/calls for
the front desk
```

设计原则

Design principles

1. **模型只"提议"，代码来"执行"。** 营业时间、提前量、防重复预约、医生和服务是否匹配，全都在 `scheduling.py` 里硬校验。模型说错时间也订不进去。写入时会再查一次冲突（slot 可能刚被别人占了）。
   The model only "proposes"; the code "executes". Opening hours, minimum notice, double-booking prevention, and whether the provider offers that service are all hard-checked in `scheduling.py`. Even if the model gets the time wrong, the booking won't go through. Conflicts are checked again at write time (the slot may have just been taken).
2. **身份绑定在通话上，不在参数里。** `verify_patient`（姓名 + 生日）成功后，patient_id 存在 `CallContext` 里；工具参数里**根本没有 patient_id**，所以来电者再怎么"说服"模型，也动不了别人的预约。

Identity is bound to the call, not passed as a parameter. After `verify_patient` (name + date of birth) succeeds, the patient_id is stored in `CallContext`; the tools **have no patient_id parameter at all**, so no matter how a caller "persuades" the model, they can't touch someone else's appointments.

3. **永远有退路。** 处理不了 → `take_message` 留言；客户要求 / 情绪不好 / 卡住两次 → `transfer_to_human` 转前台；急症 → 让对方挂了打 911（写在 prompt 安全规则最前面）。
   There's always a fallback. Can't handle it → `take_message`; caller asks for a person / is upset / gets stuck twice → `transfer_to_human` to the front desk; medical emergency → tell them to hang up and call 911 (this is the first safety rule in the prompt).
4. **不给医疗建议，不报检查结果。**
   No medical advice, no test results.
5. **说完再挂。** `end_call` / 转接不会立刻执行：等模型把"再见"说完，Twilio 回传 `mark` 事件确认音频播完了，才真正挂断或转接。
   Finish speaking before hanging up. `end_call` / transfers don't execute immediately: we wait until the model has finished saying goodbye and Twilio sends back a `mark` event confirming the audio has played, and only then hang up or transfer.
6. **打断（barge-in）。** 来电者一开口：清掉 Twilio 里还没播的音频 + `conversation.item.truncate` 告诉模型"你只说到这里"，模型就不会以为对方听到了没播出来的部分。
   Barge-in (interruptions). As soon as the caller starts talking: clear the audio Twilio hasn't played yet + send `conversation.item.truncate` to tell the model "you only got this far", so the model doesn't assume the caller heard the part that was never played.
7. **文字和语音用同一套工具。** `scripts/text_chat.py` 用同样的 prompt、工具和数据库，打字就能测完整业务流程，不用每次都打电话。
   Text and voice share the same tools. `scripts/text_chat.py` uses the same prompt, tools and database, so you can test the whole business flow by typing instead of placing a call every time.

两种电话接入方式（都已实现）

Two ways to connect the phone line (both implemented)

- **默认：Twilio Media Streams**（`twilio_stream.py`）——音频经过我们的服务器。可控、可调试，资料最多。
  Default: Twilio Media Streams (`twilio_stream.py`) — audio flows through our server. Controllable, debuggable, best documented.
- **可选：OpenAI SIP**（`openai_sip.py`）——Twilio Elastic SIP Trunk 直接连到 `sip:<project_id>@sip.api.openai.com`，OpenAI 发 `realtime.call.incoming` webhook，我们 accept 后挂一个 sideband websocket 只处理工具调用。少一跳，延迟更低，但跟 OpenAI 绑得更紧。
  Optional: OpenAI SIP (`openai_sip.py`) — a Twilio Elastic SIP Trunk connects directly to `sip:<project_id>@sip.api.openai.com`; OpenAI sends a `realtime.call.incoming` webhook, we accept the call and attach a sideband websocket that only handles tool calls. One fewer hop and lower latency, but tighter lock-in to OpenAI.

---

4. 怎么跑

How to run it

```
uv sync
```

```
cp .env.example .env
```

填好 `OPENAI_API_KEY`（Twilio 那几项先不填也行）。

Fill in `OPENAI_API_KEY` (the Twilio entries can stay empty for now).

```
uv run python -m scripts.seed
```

不用电话，先打字测：

Test by typing first, no phone needed:

```
uv run python -m scripts.text_chat
```

可以试："我是 Alice Nguyen，生日 1985-04-12，想把明天的预约改到下周三下午。"

Try: "I'm Alice Nguyen, born 1985-04-12, and I'd like to move tomorrow's appointment to next Wednesday afternoon."

跑测试：

Run the tests:

```
uv run pytest -q
```

接真电话：

Connect a real phone line:

1. 启动服务：
   Start the server: `bash uv run uvicorn app.main:app --port 8000`
2. 暴露公网 https，然后把地址填进 `.env` 的 `PUBLIC_BASE_URL`：
   Expose it over public HTTPS, then put the address in `PUBLIC_BASE_URL` in `.env`: `bash ngrok http 8000`
3. Twilio 控制台 → 买一个号码 → Voice "A call comes in" → Webhook `POST https://<你的域名>/twilio/voice`。
   Twilio Console → buy a number → Voice "A call comes in" → Webhook `POST https://<your-domain>/twilio/voice`.
4. 打这个号码。
   Call the number.

前台接口（`Authorization: Bearer <ADMIN_TOKEN>`）：`GET /admin/appointments?day=YYYY-MM-DD`、`/admin/messages`、`/admin/calls`。

Front-desk endpoints (`Authorization: Bearer <ADMIN_TOKEN>`): `GET /admin/appointments?day=YYYY-MM-DD`, `/admin/messages`, `/admin/calls`.

---

5. 代码地图

Code map

文件 File	作用 Purpose
`clinic.json`	诊所配置：时区、营业时间、休息日、服务项目和时长、医生、转接号码 Clinic config: time zone, opening hours, holidays, services and durations, providers, transfer number
`knowledge/*.md`	FAQ / 政策（诊所直接改 markdown 就行） FAQ / policies (the clinic just edits the markdown)
`app/scheduling.py`	排班规则：找空档、预约、改约、取消、患者查找/注册 Scheduling rules: find open slots, book, reschedule, cancel, look up / register patients
`app/tools.py`	给模型的 10 个工具 + 分发器 + `CallContext` The model's 10 tools + dispatcher + `CallContext`
`app/prompts.py`	system prompt（角色、语气、安全规则、流程，每通电话注入当前时间和来电号码） System prompt (role, tone, safety rules, flow; current time and caller number injected per call)
`app/realtime_agent.py`	跟传输方式无关的 Realtime 会话处理：配置、工具循环、转写 Transport-agnostic Realtime session handling: config, tool loop, transcription
`app/telephony/twilio_stream.py`	Twilio ⇄ OpenAI 音频桥、打断、说完再挂、签名校验 Twilio ⇄ OpenAI audio bridge, barge-in, finish-before-hangup, signature validation
`app/telephony/openai_sip.py`	SIP 直连方式 Direct SIP connection
`app/call_lifecycle.py`	通话记录落库 + 结束后 LLM 摘要 Persist call records + post-call LLM summary
`app/admin.py`	前台查看接口 Front-desk viewing endpoints
`scripts/text_chat.py`	打字测试 Text-based testing

文件 File	作用 Purpose
`tests/`	排班规则、工具权限、TwiML，还有一个用假 OpenAI 服务的端到端桥接测试 Scheduling rules, tool permissions, TwiML, plus an end-to-end bridge test against a fake OpenAI server

6. 模拟电话实测（2026-10-17，真的 OpenAI Realtime）

Simulated call testing (2026-10-17, real OpenAI Realtime)

`scripts/simulate_call.py`：用"假来电者"（LLM 扮演角色 + TTS 合成语音）代替 Twilio，以真实电话的格式（8kHz μ-law、实时推流、回传播放标记）打到 `/twilio/media-stream`。服务器那边走的是生产代码。每通电话在 `data/sim_calls/<场景>/` 下输出 `transcript.md` 和双声道录音 `call.wav`。

`scripts/simulate_call.py`: a "fake caller" (an LLM playing a role + TTS-synthesized voice) stands in for Twilio and calls `/twilio/media-stream` in real phone format (8kHz μ-law, real-time streaming, playback marks sent back). The server side runs the production code. Each call writes `transcript.md` and a two-channel recording `call.wav` under `data/sim_calls/<scenario>/`.

```
uv run python -m scripts.simulate_call
```

一共跑了 4 轮，前后找到并修掉的问题：

Four rounds in total; the problems found and fixed along the way:

问题 Problem	原因 Cause	修法 Fix
来电者说完以后要等 4–8 秒 agent 才开口 After the caller finished, the agent took 4–8 seconds to respond	`semantic_vad` 在电话音质下判断"说完了"太慢 `semantic_vad` is too slow to detect end-of-speech on phone-quality audio	改用 `server_vad` 600ms，延迟稳定在 1.1–2 秒；可通过 `VAD_TYPE` 配置 Switched to `server_vad` at 600 ms; latency is now a steady 1.1–2 s; configurable via `VAD_TYPE`
诊所下班了还"转真人"（转过去会没人接） Still "transferring to a human" after hours (nobody would answer)	转接前没检查营业时间 Opening hours weren't checked before transferring	`transfer_to_human` 在代码里检查营业时间，关门时让 agent 改为留言 `transfer_to_human` checks opening hours in code; when closed, the agent takes a message instead

问题 Problem	原因 Cause	修法 Fix
新患者被订给了不收新病人的 Chen 医生 A new patient was booked with Dr. Chen, who isn't accepting new patients	这条规则只写在 FAQ 里 The rule only existed in the FAQ	`clinic.json` 增加 `accepting_new_patients`，查空档和预约时由代码强制 Added `accepting_new_patients` to `clinic.json`, enforced in code for availability lookups and bookings
续药留言没有生日，前台找不到病历 Refill messages had no date of birth, so the front desk couldn't find the chart	只靠 prompt 约束，模型时灵时不灵 Relied on the prompt alone; the model followed it inconsistently	`take_message` 增加 `category`；续药、账单、检查结果三类没给生日就拒绝保存 Added `category` to `take_message`; refill, billing and test-result messages are rejected without a date of birth
中文姓名：丽 / 李 / 力 分不清，登记成了"李王" Chinese names: 丽 / 李 / 力 sound alike and got mixed up, registered as "李王"	电话里同音字没法区分 Homophones can't be told apart over the phone	改问健康卡上的英文名，逐个字母复述 Ask for the English name on the health card instead and read it back letter by letter
中文场景把 10 月说成 11 月 In Chinese calls, October was said as November	模型翻译英文月份时口误 The model slipped when translating English month names	工具同时返回数字日期 `date` Tools also return a numeric `date`
问候语没有诊所名字；一个回合里说三遍"稍等"；念出 prompt 规则 Greeting left out the clinic name; said "one moment" three times in one turn; read prompt rules aloud	prompt 写法 Prompt wording	改 prompt Revised the prompt
订完直接挂线，没问"还有别的吗" Hung up right after booking without asking "anything else?"	prompt 写法 Prompt wording	改 prompt：先问还有没有事，说完再见立刻调用 `end_call` Revised the prompt: ask if there's anything else first, then call `end_call` right after saying goodbye
先说"正在为您转接"，查完又改口"诊所关门了" Said "transferring you now", then backtracked with "the clinic is closed"	prompt 写法 Prompt wording	工具成功之后才宣布转接 Only announce the transfer after the tool succeeds

多语言（2026-10-18）

Multilingual (2026-10-18)

示例诊所的医生是编的：沈伟医生（英语、普通话、粤语）、Patel 医生（只会英语，不收新病人）、黄护士（英语、粤语），前台会英语和普通话。换成真实名单时只需要改 `clinic.json`：`name_zh`、`aliases`、`languages`、`front_desk_languages`。

The sample clinic's providers are made up: Dr. Wei Shen (English, Mandarin, Cantonese), Dr. Patel (English only, not accepting new patients), Nurse Wong (English, Cantonese); the front desk speaks English and Mandarin. To use a real roster, only `clinic.json` needs to change: `name_zh`, `aliases`, `languages`, `front_desk_languages`.

场景 Scenario	结果 Result	发现的问题 → 修法 Problem found → fix
普通话新患者点名"沈医生" New Mandarin-speaking patient asks for "沈医生" (Dr. Shen)	✅ 预约成功，前台收到英文摘要 ✅ Booked; front desk got an English summary	原来按英文名匹配，"沈医生"找不到 → `clinic.json` 增加 `aliases`，代码同时匹配中英文名 Matching was by English name only, so "沈医生" wasn't found → added `aliases` to `clinic.json`; code now matches both Chinese and English names
中英夹杂改约（老病人 Mei Lin） Reschedule in mixed Chinese/English (existing patient Mei Lin)	✅ 第二次跑通过 ✅ Passed on the second run	第一次 Mei 被听成 May，身份核对失败 → 姓 + 生日 + 来电号码一致时允许名字拼写有出入；姓名颠倒也能认出。生日不对永远不放行 First run heard "Mei" as "May" and verification failed → allow first-name spelling differences when last name + DOB + caller number all match; reversed name order is recognized too. A wrong DOB is never accepted
粤语新患者打流感针 New Cantonese-speaking patient wants a flu shot	✅ 第二次全程粤语 ✅ All Cantonese on the second run	第一次回答夹杂普通话 → prompt 明确"粤语不是普通话" First run mixed in Mandarin → prompt now states explicitly that "Cantonese is not Mandarin"
英文点名不收新病人的 Patel 医生 English caller asks for Dr. Patel, who isn't taking new patients	✅ 代码挡住，改推沈医生 ✅ Blocked in code; offered Dr. Shen instead	—
中文病人一接通先听到英文 Chinese-speaking patients heard English first when the call connected	✅	改为中英双语问候 Switched to a bilingual Chinese/English greeting

另外：给前台的摘要和留言内容一律用英文，并注明来电者说的是什么语言。 仍要注意：Realtime 的粤语明显比普通话弱，粤语病人多的话要用真人重点测；通话末尾偶尔会切回英文。

Also: summaries and messages for the front desk are always in English, noting which language the caller spoke. Still worth watching: Realtime's Cantonese is clearly weaker than its Mandarin, so with many

Cantonese-speaking patients it needs focused testing with real people; it occasionally switches back to English near the end of a call.

还没解决 / 要注意的：

Not yet solved / things to watch:

- 合成语音太干净了。真实电话有噪音、口音、对方中途停顿，ASR 准确率和 VAD 误判一定比这里差。上线前必须用真号码、真人测。
  Synthetic voices are too clean. Real calls have noise, accents and mid-sentence pauses, so ASR accuracy and VAD false triggers will definitely be worse than here. Must test with a real number and real people before launch.
- 一个回合里连着调用多个工具（核对身份 → 注册 → 查空档），每多一轮大约多 1 秒，最长 3–4 秒。可以合并工具或让模型并行调用。
  Chaining several tools in one turn (verify identity → register → check availability) adds about 1 second per round, up to 3–4 seconds. Could merge tools or have the model call them in parallel.
- 中文语音偶尔有用词错误，比如把"疫苗"说成别的词（也可能是测试脚本自己的语音识别听错了）。
  Chinese speech occasionally uses the wrong word, e.g. saying something else instead of "疫苗" (vaccine) — though it may also be the test script's own speech recognition mishearing.
- 只靠 prompt 的规则是不稳定的；凡是必须做到的，都要写进代码。
  Rules enforced only by the prompt are unreliable; anything that must always hold has to be enforced in code.

7. 真正难的地方（骨架里还没做，面试时主动提）

The genuinely hard parts (not in the skeleton yet — bring them up proactively in the interview)

1. **对接诊所现有的预约系统**——这是最大的未知数，也最容易超工期。诊所多半已经在用某个 EHR / 预约软件（加拿大常见 Jane App、OSCAR、Accuro；美国常见 Athena、eClinicalWorks、Dentrix……）。要么它有 API（把 `scheduling.py` 的 DB 读写换成调它的 API），要么没有（那就是我们的 DB 当主数据，前台在 `/admin` 上看，或者做同步）。**第一个要问的问题。**
   Integrating with the clinic's existing booking system — the biggest unknown and the most likely cause of schedule overruns. The clinic is probably already using some EHR / booking software (common in Canada: Jane App, OSCAR, Accuro; in the US: Athena, eClinicalWorks, Dentrix…). Either it has an API (swap the DB reads/writes in `scheduling.py` for calls to that API), or it doesn't (then our DB is the source of truth and the front desk views it in `/admin`, or we build a sync). **This is the first question to ask.**
2. **合规。** 美国：HIPAA，需要跟 OpenAI、Twilio 都签 BAA（两家都有，但要走企业流程）；加拿大安省：PHIPA。通话录音和转写都是 PHI：要加密存储、设保留期限、控制访问权限；录音前还要告知对方。
   Compliance. US: HIPAA — you need a BAA with both OpenAI and Twilio (both offer one, but through an enterprise process); Ontario, Canada: PHIPA. Call recordings and transcripts are PHI: they need encrypted storage, a retention period and access controls, and callers must be told before recording.

3. **姓名 / 生日的识别准确率。** 电话音质 8kHz，名字很容易听错。常见做法：让对方拼一下、复述确认，或者先用来电号码匹配候选人，再用生日确认。
   Recognition accuracy for names / dates of birth. Phone audio is 8kHz, so names are easily misheard. Common approaches: ask the caller to spell it and read it back, or first match candidates by caller number and then confirm with date of birth.
4. **延迟。** 目标是对方说完后 < 1 秒开始回应。speech-to-speech（Realtime）在这点上比 STT→LLM→TTS 三段式有优势；服务器尽量部署在离 OpenAI / Twilio 近的区域（美东）。
   Latency. The goal is to start responding < 1 second after the caller finishes. Speech-to-speech (Realtime) has an edge here over the three-stage STT→LLM→TTS pipeline; deploy the server in a region close to OpenAI / Twilio (US East).
5. **评测。** 准备一套场景脚本（预约、改约、听错名字、问医疗问题、急症、要找真人、说中文……），用 `text_chat` 的逻辑做自动回归测试；上线后每周抽查通话摘要。
   Evaluation. Prepare a set of scenario scripts (booking, rescheduling, misheard names, medical questions, emergencies, asking for a person, speaking Chinese…) and use the `text_chat` logic for automated regression tests; after launch, spot-check call summaries weekly.
6. **生产化。** SQLite → Postgres，并加排他约束保证不会重复预约；`/admin` 换成真正的登录；部署（Fly.io / Render / AWS 一台小机器就够）；监控 Twilio 和 OpenAI 的错误率。
   Production hardening. SQLite → Postgres, with an exclusion constraint to guarantee no double-booking; replace `/admin` with real login; deploy (one small machine on Fly.io / Render / AWS is enough); monitor Twilio and OpenAI error rates.
7. **外呼**（预约提醒、确认）——后续很自然的扩展，用 Twilio 发起外呼，接到同一套 agent 上。
   Outbound calls (appointment reminders, confirmations) — a natural next step: place outbound calls with Twilio and connect them to the same agent.

8. 面试时可以问客户的问题

Questions to ask the client in the interview

- 现在用什么预约 / 病历系统？有没有 API？
  What booking / medical records system do you use now? Does it have an API?
- 一天大概多少通电话？高峰在什么时候？下班后的电话现在怎么处理？
  Roughly how many calls a day? When are the peaks? How are after-hours calls handled today?
- 来电里预约、改约、问问题、要处方续药的比例大概是多少？
  Roughly what share of calls are bookings, reschedules, questions, and prescription refills?
- 需要哪些语言？
  Which languages do you need?
- 在美国还是加拿大？对合规（BAA / 录音告知）有什么要求？
  Are you in the US or Canada? What are your compliance requirements (BAA / recording disclosure)?
- "一两百个客户"指的是病人，还是你们服务的诊所数量？
  Does "one or two hundred clients" mean patients, or the number of clinics you serve?
- 转真人转到哪个号码？没人接的时候怎么办？
  Which number should calls be transferred to? What happens when nobody picks up?

参考

References

- OpenAI Realtime conversations: https://developers.openai.com/api/docs/guides/realtime-conversations
- OpenAI Realtime SIP: https://developers.openai.com/api/docs/guides/realtime-sip
- Twilio Elastic SIP Trunking × OpenAI: https://www.twilio.com/en-us/blog/developers/tutorials/product/openai-realtime-api-elastic-sip-trunking
- AI 前台架构概览 (freeCodeCamp)
  Overview of AI receptionist architecture (freeCodeCamp): https://www.freecodecamp.org/news/how-ai-receptionists-work-the-architecture-behind-ai-phone-agents/